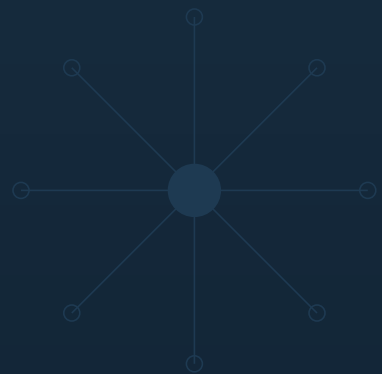


NC

Network Cloak

Software Design & Engineering Specification

SDES Version 1.0 — Complete 7-Volume Edition



Document Type:	Software Design & Engineering Specification (SDES)
Status:	Draft — Living Specification
Platforms:	Android · Windows (Linux / macOS — Future Roadmap)
Architecture:	Local-First · No Cloud · No Account Required
Version:	1.0

Local-First Privacy Firewall · No Cloud Required · User-Controlled

Volumes I–VII: Vision · Features · UI/UX · Architecture · Data · Security · Roadmap

Table of Contents

Complete overview of all seven volumes and sections

Volume I — Product Foundation & Vision

- Executive Summary
- Vision Statement
- Mission Statement
- Core Philosophy — Seven Principles
- Product Identity (Is / Is Not)
- Target User Personas
- Core Product Pillars — Six Engines
- Product Modes
- Success Metrics & KPIs

Volume II — Functional Requirements & Core Features

- Firewall Engine — Rules, Hierarchy, Actions, Lockdown, App Detection
- Shield Engine — Classification, Profiles, Trusted Networks
- Cloak Engine — LAN Exposure, Hotspot Hardening, Platform Capabilities
- DNS Guard Engine — Providers, Blocklists, Logs
- Watchtower Engine — Live Monitor, History, Bandwidth, Alerts, Export
- Rule Engine — Evaluation Pipeline, Conflict Resolution, Scheduling

Volume III — UI/UX Design Specification

- Design Philosophy & The Four-Second Rule
- Progressive Disclosure Model (Four Levels)
- Navigation Architecture
- Screen Specifications (Home, Firewall, DNS Guard, Watchtower)
- Beginner-First Language Translation Guide
- Accessibility Requirements (WCAG 2.1 AA)
- Animation & Motion Philosophy

Volume IV — Technical Architecture & Engineering Design

- Clean Architecture Overview
- Layered Architecture Diagram
- Module Dependency Map
- State Management (Riverpod)
- Android Architecture (Kotlin / VpnService)
- Windows Architecture (Rust / C# / WFP)
- Platform Abstraction Layer (PAL)
- Performance Goals & Budgets

Volume V — Data Architecture & Storage Design

- Data Architecture Philosophy
- Entity Relationship Overview
- Database Schema — 15 Tables (network_cloak.db)
- Secure Storage Mapping by Platform
- Import / Export Specification
- Data Retention Policy

Volume VI — Security, Privacy & Threat Model

- Security Philosophy & Privacy Principles
- Threat Model — Seven Actors
- Trust Boundary Architecture Diagram
- Android Required Permissions
- Windows Required Privileges
- Attack Surface Analysis
- Privacy Guarantees
- Pre-Release Security & Privacy Checklist (15 items)

Volume VII — Development Roadmap, Standards & Governance

- Development Phases 1–9 with Exit Criteria
- Coding Standards (Dart, Kotlin, Rust/C#)
- CI/CD Pipeline — Nine Stages
- Testing Strategy — Seven Levels
- Release Management Stages
- Risk Register
- Accessibility Standards
- Post-V1 Future Roadmap
- Definition of Done
- Final Product Statement

Volume I

Product Foundation & Vision

[Executive overview](#) · [Philosophy](#) · [Personas](#) · [Pillars](#) · [Modes](#) · [Metrics](#)

1. Executive Summary

Network Cloak is a local-first privacy firewall for Android and Windows. It gives users complete visibility into application network behaviour, granular control over internet access, and automatic protection on untrusted networks — all without a cloud account, hosted VPN service, or any subscription. This SDES defines the architecture, vision, requirements, and implementation guidance across seven volumes. The intended audience is an engineering team or AI coding assistant implementing the product from scratch.

Core Constraint: Network Cloak must remain fully functional offline after installation. The only permitted online operations are optional blocklist updates and optional version checks — both must be explicitly user-initiated, never automatic.

2. Vision Statement

Modern users connect to public Wi-Fi in cafés, hotels, airports, universities, and shared offices every day. The vast majority cannot discover which applications are communicating over the network, how much telemetry is collected, which trackers are contacted, or how to harden their device on an untrusted connection. Existing solutions — VPN services, enterprise firewalls, DNS blockers — are designed either for security professionals or for users willing to trust third-party infrastructure. Network Cloak provides a different model: a transparent, local-first firewall that empowers non-technical users to make informed decisions about network access without any dependence on remote servers, accounts, or fees.

3. Mission Statement

Build the most user-friendly local privacy firewall for Android and Windows that **provides strong local privacy controls, reduces unnecessary network exposure, protects users automatically on public Wi-Fi**, gives complete visibility into application network behaviour, respects user privacy by avoiding all unnecessary telemetry, operates entirely without mandatory servers, and remains approachable for non-technical users while providing advanced controls for experts.

4. Core Philosophy — Seven Principles

Every engineering decision must be evaluated against all seven principles. A feature that violates any principle must be redesigned or removed, not the principle.

Principle	Statement	Engineering Implication
Privacy First	The application exists to protect user privacy.	No feature may require unnecessary data collection or remote transmission.
Local First	Process all data locally whenever possible.	Cloud services must never be required for core functionality.
Transparency	Every action must be explainable in plain language.	All blocks, rules, and alerts must clearly state their reason.
User Control	The application suggests; the user decides.	No permanent system change without explicit user consent.
Safe Defaults	Default settings prioritise user safety.	Beginners receive meaningful protection from the very first launch.
Simplicity	Powerful capabilities, approachable interface.	Technical jargon must be hidden behind progressive disclosure.
Zero Infrastructure	No mandatory backend or subscription.	Eliminate all cloud, authentication, analytics, and relay dependencies.

Table 1 — Core Philosophy: Seven Principles

5. Product Identity

What Network Cloak Is

- A local privacy firewall — all rules enforced entirely on-device with no cloud intermediary
- A public Wi-Fi protection tool — automatic hardening when connecting to untrusted networks
- A same-network exposure reduction tool — controls local area network (LAN) visibility
- A DNS privacy manager — encrypted DNS queries with domain-level tracker blocking
- A real-time network visibility dashboard — live monitoring of all active connections
- A temporary network lockdown system — time-limited controls without permanent rule changes
- A rule-based network control platform — per-app, per-profile, per-network rules with strict priority ordering

What Network Cloak Is Not

- Not a VPN provider — no hosted relay servers, no tunnelling to remote infrastructure
- Not an anonymity tool or Tor replacement — traffic metadata remains visible to ISPs
- Not an antivirus — no file scanning, malware removal, or signature databases
- Not an enterprise endpoint management or MDM solution
- Not a man-in-the-middle HTTPS inspector — no SSL interception or content decryption

- Not a cloud security service — operates entirely without cloud connectivity

6. Target User Personas

The product serves four distinct personas. All UI and feature decisions must remain accessible to Persona 1 while satisfying the deeper needs of Personas 2–4.

Persona	Description	Primary Needs
Everyday User	Non-technical; uses public Wi-Fi occasionally; values privacy but has low technical knowledge.	Simple interface, one-tap protection, safe defaults, plain-English explanations, zero configuration required.
Privacy Enthusiast	Understands networking; wants granular control and full visibility into all traffic.	Per-app rules, DNS customisation, detailed logs, traffic analysis, data export, advanced configuration.
Student / Traveller	Frequently connects to cafés, airports, libraries, hotels, and university networks.	Automatic Public Wi-Fi Shield, trusted profiles, temporary lockdown, low battery impact.
Professional	Uses corporate networks, home office, and travels across different network environments.	Work profile, trusted office networks, strong traffic visibility, background traffic control, import/export.

Table 2 — Target User Personas

7. Core Product Pillars — Six Engines

Network Cloak is organised into six independent but tightly coordinated engines. Each engine owns a precisely defined domain. All engines communicate exclusively through the Rule Engine, which is the sole authorised policy decision point.

Engine	Primary Domain	Key Capabilities
Firewall Engine	Per-app network access control	Allow · Block · Ask · Temporary rules · Lockdown mode · Background traffic control · Priority-ordered rule hierarchy
Shield Engine	Trusted network management	Automatic network classification · Wi-Fi trust profiles · Public network hardening · Auto-profile switching
Cloak Engine	LAN exposure reduction	Local network visibility control · Hotspot hardening · Same-network service restriction · Windows profile integration
DNS Guard	DNS privacy & filtering	DNS-over-HTTPS / TLS · Tracker blocklists · Per-domain exceptions · DNS query logs · Multiple provider support
Watchtower	Network visibility & monitoring	Live connection monitor · Per-app bandwidth stats · Connection history · Anomaly alerts · CSV/JSON export
Rule Engine	Policy evaluation & coordination	Priority-ordered evaluation · Conflict resolution · Temporary rule scheduling · Profile-aware decisions · Audit logging

Table 3 — Six Engines and Their Responsibilities

8. Product Modes

Modes are presets that simultaneously configure all six engines for a specific context. Users switch modes with a single tap. Each mode has a clear security posture.

Mode	Firewall	Shield	DNS	LAN	Purpose
Home	Standard	Off	Private	Allow	Trusted home network — relaxed daily use
Work	Standard	On	Strict	Allow	Known office or corporate network
Public Wi-Fi	Strict	On	Strict	Block	Cafés, airports, hotels, transit hubs
Travel	Strict	On	Strict	Block	Unknown networks while abroad
Banking	Strict	On	Strict	Block	Secure financial transactions
Gaming	Relaxed	On	Standard	Allow	Low-latency gaming with minimal blocking
Child Safety	Strict	On	Filtered	Block	Family-safe network with content filtering
Sleep	Background	On	Private	Block	Overnight — minimal background traffic allowed
Lockdown	Block All	On	Strict	Block	Emergency — blocks everything except the allowlist
Custom	User-set	User	User-set	User-set	Fully tailored user configuration

Table 4 — Product Modes and Engine Configurations

9. Success Metrics & KPIs

Success is defined by user outcomes, not installation numbers. All metrics are measured locally without transmitting user data.

- Within two seconds of opening the app, users can state their current protection status
- Non-technical users successfully manage network access for common apps without external help
- Temporary protection modes are regularly used — demonstrating the feature is understood
- Users discover unwanted background network activity using the Watchtower screen
- Cold-start time under two seconds; battery impact acceptable on mid-range Android devices
- Full functionality verified in airplane mode — no cloud dependency exists
- Android and Windows experiences are functionally equivalent for all core features

Volume II

Functional Requirements & Core Features

Firewall · Shield · Cloak · DNS Guard · Watchtower · Rule Engine

1. Firewall Engine

The Firewall Engine is the primary enforcement layer. Every outbound network request passes through the rule evaluation pipeline before being permitted or denied. Firewall behaviour must always be predictable, transparent, reversible, and explainable in plain language to any user.

1.1 Rule Priority Hierarchy

Priority	Rule Level	Example	Scope
1 — Highest	Emergency Lockdown	Lockdown mode active	Overrides all other rules unconditionally
2	Temporary Rules	Allow 30 minutes then restore	Auto-expire at the scheduled time

Priority	Rule Level	Example	Scope
3	Session Rules	Allow until next reboot	Cleared on reboot or disconnect
4	Manual App Rules	Block Chrome permanently	User-created per-app rule
5	Profile Rules	Home profile allows all traffic	Active only when profile is selected
6	Global Rules	Block all background traffic	Applies system-wide across all apps
7 — Lowest	Default Behaviour	Ask user on new connection	Fallback when no rule matches

Table 5 — Firewall Rule Priority Levels

1.2 Rule Actions

Action	Description	Notes
Allow	Permit the connection unconditionally.	Logged as allowed.
Block	Deny the connection and log the attempt.	User sees blocked count in Watchtower.
Ask	Display an interactive permission dialog.	Default for new or unknown apps.
Temporary Allow	Permit for a defined duration, then automatically restore the previous rule.	Requires duration selection.
Temporary Block	Deny for a defined duration, then automatically restore the previous rule.	Requires duration selection.
Allow LAN Only	Permit local network access only — no internet connectivity.	Useful for printers, NAS devices, smart home.
Allow Internet Only	Permit internet access only — block local network communication.	Reduces lateral LAN exposure.
Block Background	Allow when app is in foreground; deny when the app is not active.	Reduces idle telemetry and background connections.
Restrict	Apply conditional limits based on network type, active profile, or time of day.	Advanced conditional rule logic.

Table 6 — Firewall Rule Actions

1.3 Temporary Rule Durations

Temporary rules allow users to grant or restrict access for a finite, user-selected period without permanently modifying their rule set. Available durations: 5 minutes · 10 minutes · 30 minutes · 1 hour · 3 hours · 6 hours · Until screen locks · Until device unlocks · Until disconnected · Until Wi-Fi changes · Until device reboots · Custom duration.

Example: "Allow Chrome — 30 minutes — then restore previous rule automatically." The user gets friction-free temporary access without creating a permanent exception. A notification confirms when the timer expires and the previous rule is restored.

1.4 Lockdown Mode

One-tap emergency protection. Lockdown immediately blocks all applications except the configured Lockdown Allowlist. All background traffic is disabled. Strict DNS filtering is enforced. LAN access is blocked on all non-Trusted networks. A persistent status-bar indicator is shown for the full duration. A clear, always-visible one-tap exit is provided. Default Lockdown Allowlist: Phone calls, Emergency services, Banking apps, Messaging apps, Authenticator apps, Navigation / Maps, and VoIP if configured.

1.5 Application Detection

On Android, applications are identified by their OS-assigned UID mapped to package names (e.g. com.android.chrome) and display names (e.g. Chrome). System applications are clearly labelled [System]. Unknown UIDs are labelled [Unknown Process] and receive the Ask default. On Windows, connections are mapped from PIDs to executable paths using the Windows process enumeration API. Both platforms display the app icon alongside its display name.

2. Shield Engine

The Shield Engine automatically detects, classifies, and responds to network environments, removing the burden of manual network assessment from the user by automatically applying the appropriate protection level for every network the device connects to.

2.1 Network Classification States

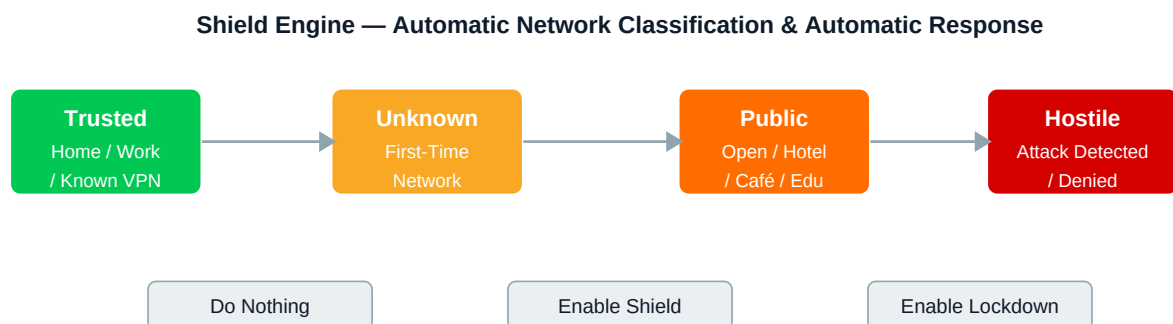


Figure 1 — Shield Engine Network States and Automatic Responses

2.2 Unsafe Network Detection Signals

- Open authentication — no password or encryption required to join the network
- SSID not in the user's Trusted Network list — first-time or unrecognised network
- Weak encryption detected — WEP, WPA-TKIP, or completely open network
- Captive portal detected on initial connection — common in hotels and airports
- Security protocol downgrade attempt observed during connection negotiation
- BSSID mismatch against a previously trusted SSID — possible evil-twin attack indicator
- Rapid or unexpected network change — possible hotspot switching

2.3 Trusted Network Profiles

A Trusted Network is a network the user has explicitly marked as safe. Each trusted network can be associated with a specific Profile (e.g. Home, Work). When the device connects to a Trusted Network, the Shield Engine automatically activates the associated profile and applies its protection level. The user is always notified of profile switches.

3. Cloak Engine

The Cloak Engine reduces a device's unnecessary exposure to other devices on the same local area network. On public or unknown networks many devices inadvertently broadcast discoverable services. Cloak makes these services less accessible without breaking required connectivity.

Capability	Android	Windows
Network Discovery blocking	Via VpnService routing rules	Via Windows Firewall Public Profile toggle
File Sharing warning	N/A — not applicable	Detect active File Sharing; warn user; offer to disable
SMB / NetBIOS blocking	N/A — not applicable	Block ports 137–139, 445 on untrusted networks via WFP
mDNS / Bonjour restriction	Partial via VPN rules	Via Windows Firewall mDNS rule
Hotspot client hardening	Client isolation warning + deny-LAN rules	N/A
LAN access control	Allow / Block / Allow-LAN-Only rules	Allow / Block via WFP per-application rules

Table 7 — Cloak Engine Capabilities by Platform

Constraint: Cloak must never break required connectivity such as streaming devices, printers, or smart home hubs that the user explicitly permits. Every restriction is individually toggleable and the user can always whitelist specific local addresses.

4. DNS Guard Engine

DNS Guard improves DNS privacy by enforcing encrypted DNS protocols and blocking unwanted domains at the DNS resolution stage. It supports multiple providers, configurable blocklists, per-domain exceptions, and a full DNS query log.

Provider	Protocol	Primary Endpoint	Focus
Cloudflare	DoH / DoT	1.1.1.1 — one.one.one.one	Speed and privacy
Cloudflare Family	DoH / DoT	1.1.1.3 — family.cloudflare-dns.com	Malware + adult content filtering
Google	DoH / DoT	8.8.8.8 — dns.google	Reliability
Quad9	DoH / DoT	9.9.9.9 — dns.quad9.net	Security and malware blocking
NextDNS	DoH / DoT	User-configured endpoint	Highly configurable filtering
AdGuard DNS	DoH / DoT	94.140.14.14	Ad and tracker blocking
Custom	DoH / DoT	User-defined URL or IP	Any RFC-compliant resolver

Table 8 — Supported DNS Providers

Category	What It Blocks	Default State
Advertising	Ad networks, banner and video ad servers, ad exchanges	Balanced (moderate)

Category	What It Blocks	Default State
Analytics	User behaviour tracking, session analytics, A/B testing platforms	Balanced (moderate)
Telemetry	OS and application diagnostic and crash data collectors	Balanced (moderate)
Tracking	Cross-site, cross-app, and device fingerprinting trackers	Balanced (moderate)
Malware	Known malicious domains and command-and-control servers	On — always enforced
Phishing	Social engineering sites and credential theft infrastructure	On — always enforced
Cryptomining	Browser-based and script-injected cryptomining	On — always enforced
Ransomware	Known ransomware distribution and C2 endpoints	On — always enforced
Adult Content	Adult material — parental control feature	Off — user opt-in only
Social Media	Social platform domains — focus/study mode feature	Off — user opt-in only
Custom Allow	User-defined domains that bypass all blocklists	User-defined
Custom Block	User-defined domains that are always denied	User-defined

Table 9 — DNS Guard Blocklist Categories

5. Watchtower Engine

Watchtower is the visibility engine. It gives users a clear, real-time view of all network activity. Watchtower does not intercept or inspect packet contents — it monitors connections at the metadata level (application, destination, port, protocol, bytes).

Feature	Description	Notes
Live Connection Monitor	Real-time list of all active connections with app name, destination host, port, and protocol.	Refreshed per-event from native layer
Connection History	Searchable, filterable log of past connections with timestamps and rule decisions.	Stored in connection_history table
Per-App Bandwidth	Bytes sent and received per application for current session and all-time history.	Aggregated from packet counts
DNS Query Log	Every DNS query resolved or blocked, with category label and response latency.	Stored in dns_logs table
Blocked Traffic Summary	Count and breakdown of all blocked connections by app, category, and time period.	Derived from firewall logs
Anomaly Alerts	Notification when an app contacts an unusually high number of unique destinations.	Configurable threshold
Network Statistics	Total data transferred, connection counts, and block rates across all time periods.	application_stats table
Export	Export connection history and DNS logs to CSV or JSON for external analysis.	Local file — never uploaded

Table 10 — Watchtower Engine Features

Privacy Note: Watchtower data is stored locally only and is never transmitted to any server. Users can clear all history instantly. Retention periods are user-configurable. Export always produces a local file — there is no upload option.

6. Rule Engine — Evaluation Pipeline

The Rule Engine is the central decision-making system. Every subsystem submits connection events to the Rule Engine, which evaluates rules in strict priority order, resolves all conflicts, logs the decision with its full justification, and returns a final action.

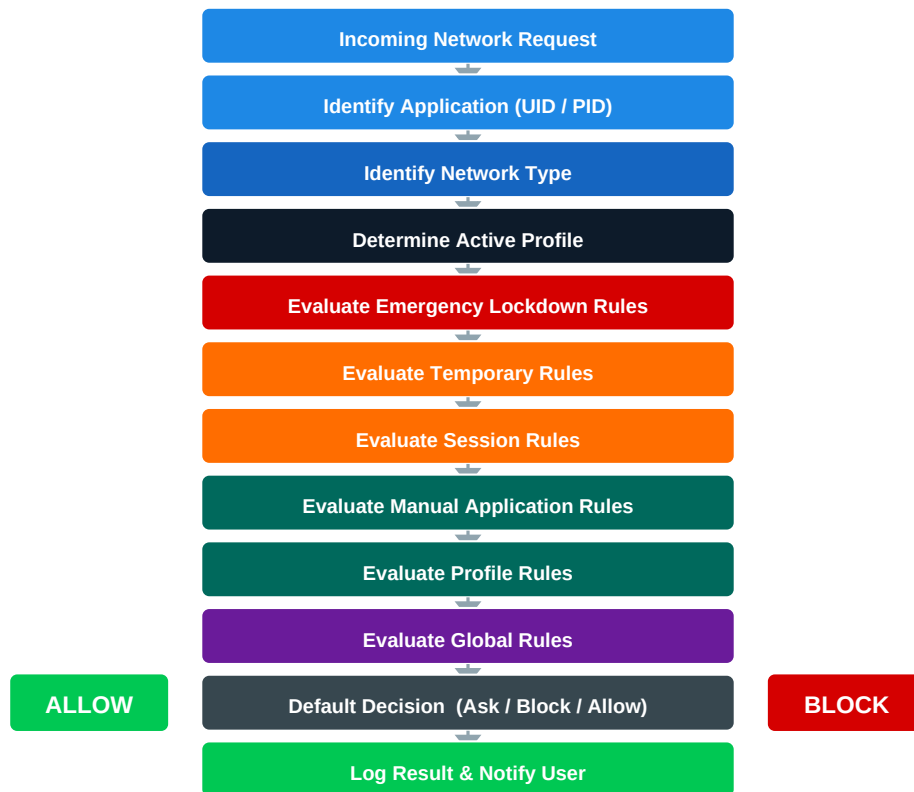


Figure 2 — Rule Engine Evaluation Pipeline

6.1 Conflict Resolution Rules

- Higher-priority rules always override lower-priority rules — no exceptions permitted
- Among rules at the same priority level, the most recently created rule wins
- A Block rule always overrides an Allow rule at the same or lower priority level
- Lockdown (Priority 1) overrides all rules including user-created manual rules
- Temporary rules (Priority 2) override permanent manual rules until they expire
- Profile switching changes which Profile Rules are active — it never deletes rules
- All decisions are logged with the rule that triggered them and its priority level

UI/UX Design Specification

Design philosophy · Disclosure model · Screens · Language · Accessibility

1. UI/UX Philosophy

Network Cloak's interface must be the opposite of a traditional firewall utility. Most tools immediately expose ports, protocols, and IPs without context. Network Cloak must feel like a modern OS security centre: calm, professional, and instantly readable by a non-technical user. Complexity exists but is never shown until the user actively seeks it.

The Four-Second Rule: Within two seconds of opening the application, a user must be able to answer all four questions without reading: (1) Am I protected? (2) What network am I on? (3) What is happening right now? (4) What should I do next? If the UI cannot answer all four questions immediately, it must be redesigned.

2. Progressive Disclosure Model

Level	Triggered By	Information Visible	Audience
1	Immediate — always visible	Protected / Not Protected indicator · Network name · Mode name · Block count	All users
2	Single tap on a connection or alert	Application name · Rule applied · Network type · Action taken · Timestamp	Curious users
3	Navigate to Details view	IP address · Port · Protocol · DNS query · Latency · Process name	Privacy enthusiasts
4	Navigate to Advanced Settings	Full rule editor · Raw log viewer · DNS diagnostics · Import/export · Debug mode	Power users

Table 11 — Progressive Disclosure Levels

3. Navigation Architecture

Android: five-tab bottom navigation bar always visible. Windows: persistent left-sidebar panel. Every screen reachable within three taps or clicks from Home. Deep-linking from notification actions must be supported.

Tab / Section	Primary Content
Home	Protection status · Active mode · Current network · Quick actions · Recent alerts · Live pulse
Firewall	Application list with rule status · Quick allow/block · Rule editor access
Activity	Live connection monitor · DNS log · Per-app bandwidth · Connection history · Alerts
Rules & Profiles	Profile manager · Rule editor · Trusted networks · Scheduled rules · Import/export
Settings	DNS Guard · Shield config · Cloak config · Appearance · Notifications · Advanced · About

Table 12 — Navigation Sections

4. Screen Specifications

4.1 Home Dashboard

- Large unambiguous status indicator: Protected (green) · Not Protected (red) · Partial (amber)
- Connected network name, type (Home / Work / Public / Unknown), and trust classification
- Current mode badge with single-tap mode switcher covering all ten modes
- Most recently blocked connection summary — app name, destination, time, and action
- One-tap Lockdown button — always visible and prominently placed
- Last three security alerts with tap-to-detail navigation
- Live traffic pulse animation showing real-time network activity (respects reduced-motion setting)

4.2 Firewall Screen

Lists all installed applications with current network status. Each entry shows: app icon, display name, current rule action (Allow / Block / Ask), connection count, and quick-action buttons. Tapping an application opens the Rule Editor for that app. A filter bar allows filtering by rule type, connection status, or search by name.

4.3 DNS Guard Screen

Shows active DNS provider, total queries resolved, total blocked, and top blocked domains. Users can switch DNS profile, configure blocklist categories, add custom allow / block entries, and view the full paginated DNS query log.

5. Beginner-First Language Guide

Technical Message (Never Show)	User-Friendly Message (Always Show)
Blocked outbound TCP to 142.250.193.14:443	Chrome tried to connect to Google. Blocked by your Public Wi-Fi profile.
Error 504 — Gateway Timeout	Could not update the DNS blocklist. Check your internet connection. [Retry]
mDNS broadcast suppressed on wlan0	Network Discovery is now hidden on this public Wi-Fi network.
Rule conflict: allow(priority=4) overridden by block(priority=1)	Lockdown is active. All other rules are currently overridden.
UID 10142 exceeded anomaly threshold (42 unique destinations in 60s)	Chrome is connecting to an unusually high number of servers. Tap to review.
BSSID mismatch for SSID 'CoffeeShop_Free'	Warning: This Wi-Fi network looks different from the one you trusted before. Tap to review.

Table 13 — Beginner-First Language Translation Guide

6. Accessibility Requirements (WCAG 2.1 AA)

- Screen readers fully supported: TalkBack on Android, Narrator on Windows — all interactive elements require content descriptions

- Large text and system font scaling supported up to 200% without layout breakage or truncation
- High contrast mode and colour-blindness-safe indicators — no information conveyed by colour alone
- Keyboard navigation with clearly visible focus rings and logical focus order (Windows)
- Touch targets minimum 48×48dp on Android; minimum 44×44px on Windows
- Reduced motion mode — no auto-playing animations when system reduced-motion setting is active
- All icons paired with visible text labels on the primary interface
- Error messages describe both what went wrong and the action the user should take

Volume IV

Technical Architecture & Engineering Design

Clean Architecture · Flutter · Android (Kotlin) · Windows (Rust/C#) · PAL

1. Architecture Overview

Network Cloak follows Clean Architecture with strict, enforced modular boundaries. Business logic is platform-independent. Platform-specific networking code is fully isolated behind the Platform Abstraction Layer (PAL). The UI layer never calls native networking code directly. Every module has a single, well-defined responsibility.

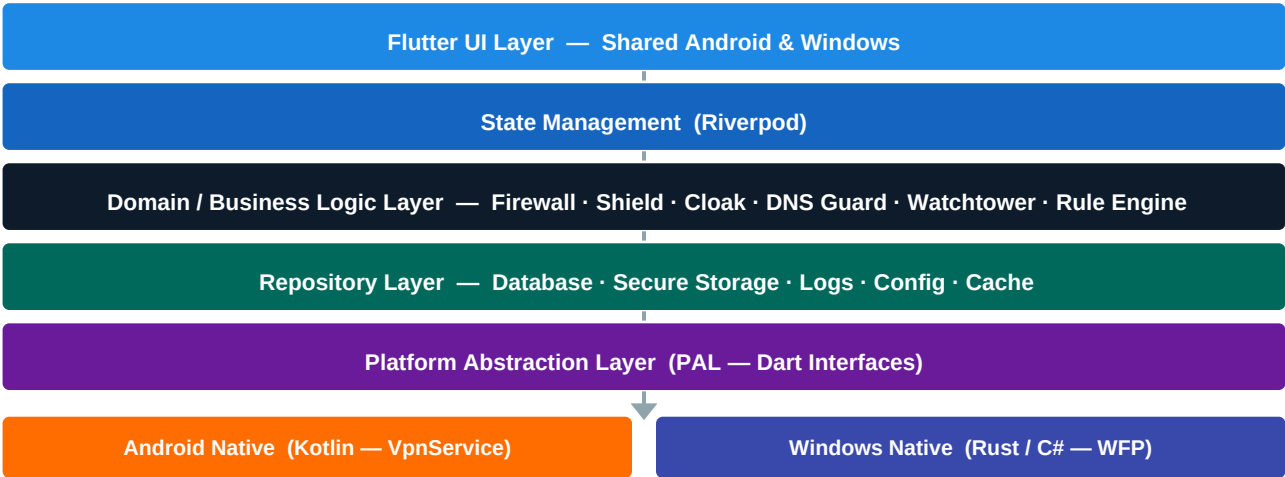


Figure 3 — High-Level Layered Architecture (Flutter UI → Domain → PAL → Native)

2. Architectural Layers

Dependency flow is strictly one-directional: Presentation → Domain → Repository → Platform → Native. No layer may import or call a layer above it.

Layer	Technology	Responsibilities	Must Never
1 — Presentation	Flutter (Dart)	UI widgets, navigation, animations, view state	Call native code directly, read database, evaluate rules
2 — Domain	Pure Dart (no Flutter dependencies)	Business rules, use cases, rule engine evaluation, entities	Import Flutter packages, access storage, reference platform APIs
3 — Repository	Dart + Drift / SQLite	Data access, configuration, cache, import/export logic	Contain business logic, reference UI, call native networking

Layer	Technology	Responsibilities	Must Never
4 — PAL	Dart abstract interfaces	Define contracts between Flutter and native platform code	Contain implementations or reference platform-specific types
5 — Native	Kotlin / Rust / C#	OS networking, VpnService, WFP callouts, process APIs	Contain business logic, UI code, or direct database access

Table 14 — Architectural Layers and Constraints

3. Module Dependency Map

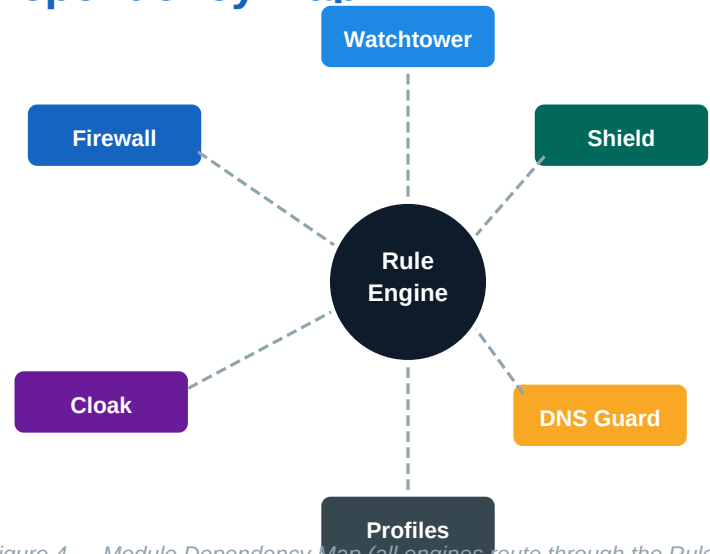


Figure 4 — Module Dependency Map (all engines route through the Rule Engine)

4. State Management

The application uses Riverpod for state management. All state changes flow through a unidirectional stream: User Action → Use Case → Repository → Notifier → UI rebuild. No mutable global state. Background state from the native layer is communicated to Flutter via EventChannel streams.

State Category	Implementation	Persistence	Update Frequency
Firewall Rules	AsyncNotifierProvider	SQLite (permanent)	On rule change
Active Profile	NotifierProvider	SharedPreferences	On profile switch
Live Connections	StreamProvider	In-memory only	Per-connection event
Network Status	StreamProvider	In-memory only	On connectivity change
Temporary Rules	AsyncNotifierProvider	SQLite + memory	Per-timer tick and on change
DNS Log	AsyncNotifierProvider	SQLite (rolling)	Per DNS query
App Statistics	FutureProvider	SQLite (aggregate)	Periodic and on demand
Alert Queue	NotifierProvider	SQLite (rolling)	On alert creation

Table 15 — State Management Categories

5. Android Architecture

The Android native layer is written in Kotlin. It implements VpnService to create a local VPN tunnel through which all device traffic is routed for inspection and filtering.

Component	Responsibility
VpnService Wrapper	Manages the local VPN tunnel lifecycle: start, stop, reconnect, and graceful teardown.
Firewall Engine	Evaluates per-UID packet decisions using the priority-ordered rule set; delegates to the Rule Engine.
DNS Engine	Intercepts all DNS queries; applies blocklists; forwards permitted queries to the configured DoH/DoT provider.
Connectivity Monitor	Listens for network changes via ConnectivityManager; triggers Shield Engine profile evaluation on every change.
Rule Bridge	Translates Dart-domain rule objects into native packet filter objects used by the VpnService.
Notification Manager	Delivers security alerts, temporary rule expiry notices, Shield activations, and Lockdown notifications.
WorkManager Jobs	Handles scheduled tasks: rule expiry cleanup, blocklist refresh, statistics aggregation, log rotation.
Platform Channel	Bidirectional bridge: Dart sends commands via MethodChannel; Kotlin sends events via EventChannel.

Table 16 — Android Native Components

6. Windows Architecture

The Windows native layer is preferably written in Rust for memory safety and performance. C# is an acceptable alternative. The Windows Filtering Platform (WFP) provides kernel-mode packet filtering. A Windows Service maintains protection independently of the Flutter UI process.

Component	Responsibility
WFP Adapter	Registers callout drivers and permit/block filter conditions with the Windows Filtering Platform.
Firewall Controller	Translates Dart-domain rule objects into WFP filter GUIDs; manages rule lifecycle (add, update, remove).
Background Service	Windows Service that persists rules and enforces policy independently of the UI process lifecycle.
System Tray Agent	Minimal system-tray UI: protection status indicator, one-click Lockdown, and recent alert display.
DNS Manager	Configures Windows DNS settings; routes queries to the user-selected DoH/DoT provider.
Process Enumerator	Maps active process IDs to executable paths using Windows APIs for per-app rule enforcement.
IPC Bridge	Named pipe or local socket communication between the Windows Service and the Flutter UI process.

Table 17 — Windows Native Components

7. Performance Goals

Metric	Target	Rationale
Application cold start	< 2 seconds	Users need immediate protection awareness on launch
Rule evaluation latency	< 5 ms per decision (average)	Must not add perceptible latency to network requests
UI frame rate	60 FPS minimum on target hardware	Smooth animations maintain perception of reliability
Memory usage (Android)	< 80 MB heap in steady state	Must not displace foreground apps on 3 GB RAM devices
Background CPU (Android)	< 2% average when idle	Battery impact must be acceptable for always-on operation
Database query time	< 50 ms for common queries	Rule lookups must not block the firewall decision path
Large rule set startup	< 3 seconds with 500+ rules	Performance must remain acceptable for power users

Table 18 — Performance Goals

Volume V

Data Architecture & Storage Design

SQLite · Drift ORM · Secure Storage · Import/Export · Retention Policy

1. Data Architecture Philosophy

Every piece of data generated by Network Cloak belongs exclusively to the user. No data is transmitted without an explicit user action. The storage layer must support offline operation, encrypted storage for sensitive values, high performance on constrained hardware, forward-compatible schema migrations with rollbacks, lossless import/export, and complete data deletion on user request.

2. Entity Relationship Overview

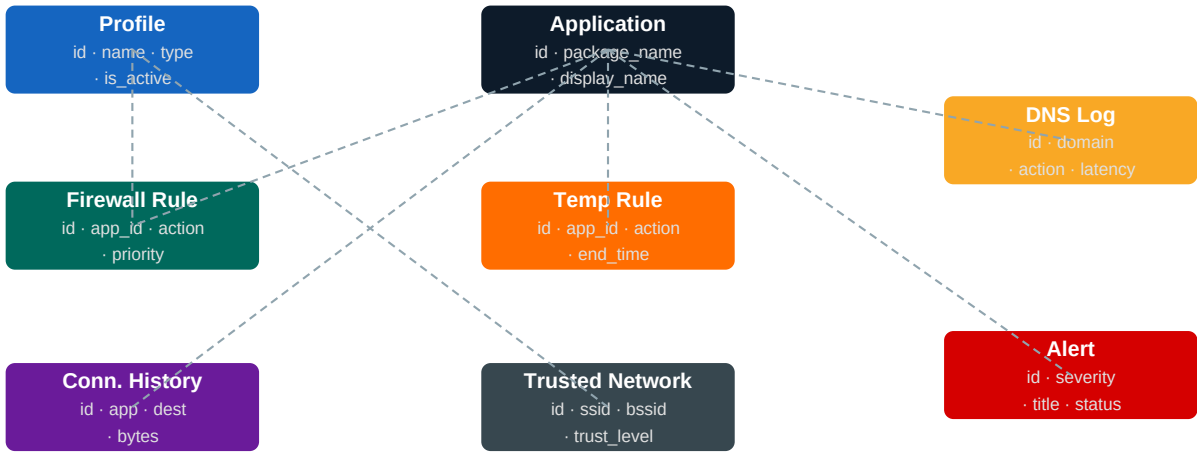


Figure 5 — Core Entity Relationships (simplified ER diagram)

3. Database Schema

All non-sensitive persistent data is stored in a single SQLite database (network_cloak.db) managed through the Drift ORM. The schema is versioned. Every migration includes a forward migration, a rollback migration, validation assertions, and automated tests.

Table	Category	Key Fields	Retention
settings	System	category, key, value, value_type, updated_at	Until changed
profiles	Permanent	id, name, type, is_system, is_active, config_json	Until deleted
firewall_rules	Permanent	id, app_id, action, priority, conditions_json	Until deleted
temporary_rules	Temporary	id, app_id, action, start_at, end_at, previous_rule_id	Auto-expire
session_rules	Session	id, app_id, action, session_id	Cleared on reboot
trusted_networks	Permanent	id, ssid, bssid, trust_level, profile_id	Until deleted
dns_profiles	Permanent	id, name, provider, protocol, endpoint, blocklists	Until changed
dns_blocklists	Permanent	id, name, category, enabled, url, checksum, updated_at	Until removed
dns_logs	Rolling	id, domain, action, category, latency_ms, app_id, timestamp	Configurable
applications	Permanent	id, package_name, display_name, version, first_seen	Until uninstalled
application_stats	Rolling	id, app_id, connections, blocked, bytes_sent, bytes_recv	Configurable
connection_history	Rolling	id, app_id, dest_host, dest_ip, port, protocol, rule_id, bytes	Configurable
alerts	Rolling	id, type, severity, title, body, app_id, status, created_at	Configurable
live_connections	Runtime	id, app_id, dest, protocol, started_at, bytes	Session only
schema_version	System	version, applied_at, description	Permanent

Table 19 — Database Schema (network_cloak.db)

4. Secure Storage

Sensitive data must never be stored in SQLite or plain-text files.

Data Type	Android Secure Storage	Windows Secure Storage
Database encryption key	Android Keystore (AES-256-GCM)	DPAPI-protected binary blob
Export file password	Android Keystore (derived key)	Windows Credential Manager
Custom DNS tokens	EncryptedSharedPreferences	Windows Credential Manager
Future auth tokens	Android Keystore (SecretKey)	DPAPI / CNG store
TLS certificates	Android Keystore (KeyPair)	Windows Certificate Store (CNG)

Table 20 — Secure Storage Mapping by Platform

5. Import / Export

Export Type	Format	Included Data	Encryption
Full Backup	JSON (zipped)	All rules, profiles, trusted networks, DNS profiles, settings	Optional AES-256
Rules Only	JSON	Firewall rules and application mappings only	Optional
DNS Configuration	JSON	DNS profiles and custom blocklist entries	Optional
Trusted Networks	JSON	All trusted network entries	Optional
Connection Log	CSV or JSON	Connection history filtered by date range	None — plain export
DNS Log	CSV or JSON	DNS query log filtered by date range	None — plain export

Table 21 — Import / Export Specification

6. Data Retention Policy

Data Category	Default Retention	Configurable Options	Auto-Delete
Connection history	30 days	7 / 30 / 90 / 365 / Forever	Yes, nightly job
DNS query logs	30 days	7 / 30 / 90 / 365 / Forever	Yes, nightly job
Application stats	90 days	30 / 90 / 365 / Forever	Yes, nightly job
Alert history	30 days	7 / 30 / 90 / 365 / Forever	Yes, nightly job
Crash logs	30 days (local)	7 / 30 / 90	Yes, nightly job
Firewall rules	Permanent	Manual delete only	No
Temporary rules	Deleted on expiry	History log toggle	Yes, on expiry
Session rules	Deleted on reboot	Not configurable	Yes, on reboot
Exported files	User-managed	N/A	No — user controls

Table 22 — Data Retention Policy

Security, Privacy & Threat Model

Threat actors · Trust boundaries · Permissions · Privacy guarantees · Checklist

1. Security Philosophy

Foundational Principle: Security is not a feature. It is the foundation upon which every feature is built. Every engineering decision must be evaluated from four mandatory perspectives: Privacy · Security · Transparency · User Control. A feature that compromises any of these without compelling justification must be redesigned or removed — not the principle.

Privacy Principles

- Data Minimisation — collect only data strictly necessary for the feature to function
- Local Processing — all data processing occurs on-device; no cloud intermediary
- User Ownership — the user owns all data; the application is only a tool to manage it
- Explicit Consent — no data is transmitted, shared, or exported without explicit user action
- Deletability — the user can delete all data at any time, immediately and completely
- No Tracking — no analytics, no telemetry, no crash reporting without explicit user opt-in

2. Threat Model

Threat Actor	Attack Vector	Mitigations	In Scope?
Casual Wi-Fi neighbour	Discover device services; access file shares; scan LAN.	Cloak LAN blocking; Shield auto-activation; mDNS restriction.	Yes — primary threat
Curious network admin	Monitor DNS if unencrypted; inspect traffic metadata.	DNS Guard enforces DoH/DoT; encrypted DNS prevents query snooping.	Yes — partial
Malicious hotspot operator	MITM on unencrypted traffic; captive portal abuse; DNS hijack.	Shield detects captive portals; DNS Guard uses encrypted DNS; user warned on detection.	Yes — primary threat
Stalkerware / malware on device	Bypass firewall rules; read logs; tamper config; exfiltrate data.	Rule integrity checks; secure storage for keys; import validation.	Partial — no kernel guarantee
Physical attacker (unlocked device)	View logs; modify rules; export configuration.	App lock (roadmap); encrypted export; secure storage.	Partial — future feature
ISP / government	Traffic metadata even with DoH; timing analysis.	Out of scope — not an anonymity tool.	No — out of scope
Nation-state / kernel exploit	Full device compromise including VPN bypass.	Outside application scope — cannot defend against kernel-level attackers.	No — out of scope

Table 23 — Threat Model: Actors, Vectors, Mitigations, and Scope

3. Trust Boundary Architecture

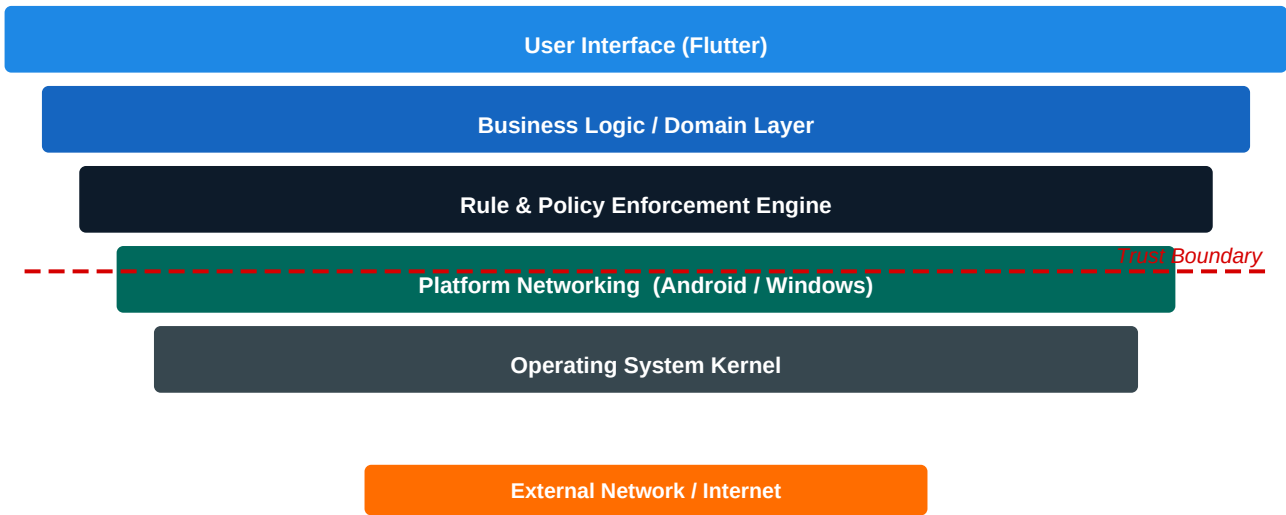


Figure 6 — Trust Boundaries (red dashed line separates trusted from untrusted zones)

4. Android Required Permissions

Permission	Required For	Justification
BIND_VPN_SERVICE	Local VPN tunnel for traffic filtering	Core firewall — without this, no per-app rules are possible on Android
FOREGROUND_SERVICE	Keep VPN running in foreground	Prevents Android from killing the firewall when the app is backgrounded
RECEIVE_BOOT_COMPLETED	Auto-start protection after reboot	Ensures rules are enforced immediately after device restarts
ACCESS_NETWORK_STATE	Detect network changes for Shield Engine	Required to trigger profile switching when the network changes
ACCESS_WIFI_STATE	Read SSID and BSSID for trusted networks	Required to identify which trusted network the device is connected to
INTERNET	Blocklist updates and DoH/DoT queries	Required for optional blocklist refresh and encrypted DNS operation
POST_NOTIFICATIONS (Android 13+)	Security alerts and rule expiry notices	Required to show block notifications, rule expiry alerts, and Shield activations
USE_BIOMETRIC (future)	Biometric app lock (roadmap feature)	Reserved — not requested until the feature is implemented

Table 24 — Android Required Permissions

5. Windows Required Privileges

Privilege / Right	Required For	Notes
Administrator (install-time only)	Install WFP callout driver; register Windows Service	Elevation required only during installation — not during normal runtime
Windows Service (LocalService)	Run background enforcement service	Service runs with reduced privileges after installation

Privilege / Right	Required For	Notes
WFP FWPM_SESSION rights	Add, modify, remove WFP filter conditions	Granted to the service account during the installation process
DNS configuration rights	Set system DNS resolver to DoH/DoT provider	May require Administrator or Network Configuration Operators group membership
Process enumeration access	Map PIDs to executables for per-app rules	Standard user rights on modern Windows — no elevation required at runtime

Table 25 — Windows Required Privileges

6. Attack Surface Analysis

Surface	Risk	Mitigation
Import file parsing — malformed JSON could inject rules or crash the app.	High	Strict schema validation; reject on any error; sandboxed parser; no partial imports
Platform channel — commands between Dart and native layers.	Medium	Input validation on both sides; whitelist of allowed actions; no raw shell execution
Blocklist download — downloaded content becomes active DNS rules.	Medium	Checksum verification; HTTPS only; user-controlled provider; no auto-execute on download
Database access — rule storage in SQLite.	Low	Drift ORM with parameterised queries; no raw SQL strings constructed from user input
Notification action tokens — tap Allow/Block from notification.	Low	Action tokens are cryptographically validated; each token is single-use only

Table 26 — Attack Surface Analysis

7. Privacy Guarantees

- No mandatory user account required at any stage of the product lifecycle
- No mandatory cloud connectivity required for any core functionality
- No automatic analytics collection, usage telemetry, or crash reporting
- No selling, sharing, or licensing of user data to any third party under any circumstances
- No automatic upload of logs, connection history, DNS logs, or statistics
- All exported files are local by default — the user explicitly chooses where to send them
- Crash reports stored locally and shared only through an explicit user action

8. Pre-Release Security & Privacy Checklist

Check	Category	Validation Method
Firewall rules enforced for all action types	Functional	Integration tests with network traffic capture
Rule priority order deterministic across all scenarios	Functional	Unit tests with known inputs at every priority level

Check	Category	Validation Method
Temporary rule expiry triggers at correct time	Functional	Timer-based integration tests
DNS filtering blocks all configured categories	Functional	DNS query tests against known blocklist domains
No sensitive data appears in log files	Privacy	Automated log content audit for IPs and domains
Secure storage read/write on all target API levels	Security	Keystore / DPAPI functional tests on all targets
Import rejects malformed and malicious files	Security	Fuzzing and malformed schema test suite
Export checksums validate on re-import	Security	Round-trip export-import test for all export types
No debug flags or logging active in production build	Security	Static analysis and build configuration review
No hardcoded credentials or API keys	Security	Secret scanning (Gitleaks) on all commits
No unnecessary permissions in manifests	Privacy	Manifest review and permission diff against baseline
Zero analytics or telemetry active in production	Privacy	Network traffic inspection of release build
Full functionality preserved in airplane mode	Functional	Airplane mode regression test for all core features
Lockdown blocks all non-allowlist traffic	Security	Traffic capture during Lockdown activation and duration
Profile switching applies correct rules immediately	Functional	Automated profile switch and rule verification test

Table 27 — Pre-Release Security & Privacy Checklist

Volume VII

Development Roadmap, Engineering Standards & Governance

Phases 1–9 · Coding standards · CI/CD · Testing · Risk Register · Roadmap

1. Development Phases

The product is implemented in nine structured phases. Each phase delivers a fully functional, tested increment. No phase begins until the previous has passed all its acceptance criteria. Partially implemented features are not permitted — every feature shipped must be complete and production-quality.

Phase	Name	Key Deliverables	Exit Criteria
1	Foundation	Project structure, theme, navigation, DI, DB schema, PAL interfaces	Architecture validated; all layers compile; navigation functional; DB migrations pass

Phase	Name	Key Deliverables	Exit Criteria
2	Firewall Engine	Rule engine, app detection, allow/block/ask, temporary rules, rule editor UI	Per-app rules enforced; temporary rules expire correctly; priority order verified
3	Shield Engine	Network detection, trusted networks, auto profile switching, Shield UI	Public Wi-Fi correctly classified; profiles switch on network change
4	Cloak Engine	LAN restrictions, hotspot hardening, Cloak UI, Windows profile integration	LAN blocking functional on Android; Windows hardening warnings displayed
5	DNS Guard	DoH/DoT providers, blocklists, per-domain exceptions, DNS log UI	Encrypted DNS active; tracker domains blocked; DNS log populated correctly
6	Watchtower	Live connections, history, per-app bandwidth, alerts, export	Real-time activity visible; all export formats working; anomaly alerts firing
7	UX Polish	Animations, onboarding wizard, accessibility, beginner language, dark mode	WCAG 2.1 AA compliance verified; 60 FPS on reference hardware; onboarding validated
8	Platform Optimisation	Battery, memory, startup time, large rule set, background performance	Cold start < 2 s; background CPU < 2%; memory within budget on mid-range Android
9	Release Candidate	Bug triage, security review, regression suite, beta feedback	Zero known critical defects; all checklists passed; store listing ready

Table 28 — Development Phases with Exit Criteria

2. Coding Standards

Dart / Flutter

- Follow the official Dart Style Guide; use dart format for all automated formatting
- Strict dart analyze with no exceptions — all lints must pass in CI
- No dynamic types — all public APIs must have complete type signatures
- No direct platform channel calls from the UI layer — all calls via Repository or PAL
- Every public class and method must have a dartdoc comment
- Use const constructors everywhere possible; prefer early returns over deep nesting
- State management: Riverpod exclusively — no setState except in isolated demo components

Kotlin (Android)

- Kotlin idioms preferred over Java compatibility constructs throughout
- Coroutines for all asynchronous operations — no raw threads or callback nesting
- Sealed classes for all platform channel command and event types
- No direct Android Context stored in ViewModel or UseCase — use dependency injection

Rust / C# (Windows)

- Rust: all unsafe blocks documented with a safety justification comment
- Rust: no unwrap() in production code paths — use ? operator and proper error propagation
- C#: use async/await consistently; never use Task.Result or Task.Wait()

- WFP callout code tested against all supported Windows versions (10 22H2, 11)

3. CI/CD Pipeline

Every commit to any branch triggers the full pipeline. No pull request merges unless all stages pass. The pipeline is the enforcement mechanism for all quality standards.



Figure 7 — CI/CD Pipeline (all stages run on every commit)

Stage	Tools	Failure Action
Format	dart format --check; ktlint; rustfmt --check	Block merge — formatting is non-negotiable
Lint	dart analyze (strict); Android Lint; Clippy (Rust)	Block merge — all lint warnings are errors
Unit Tests	flutter test; JUnit; Rust test	Block merge — 100% of unit tests must pass
Integration Tests	flutter integration_test; Android Espresso	Block merge — all integration tests must pass
UI Tests	flutter integration_test (golden); Accessibility Scanner	Block merge — no accessibility regressions
Security Scan	Gitleaks (secrets); OWASP dependency check	Block merge — no secrets or critical CVEs
Build	flutter build; Gradle; cargo build; dotnet build	Block merge — all build targets must succeed
Package	APK / AAB signing; MSIX creation	Artifacts archived and available for testers

Table 29 — CI/CD Pipeline Stages

4. Testing Strategy

Level	Scope	Tooling	Priority
Unit Tests	Rule engine evaluation; DNS filtering; profile switching; utilities	flutter test; JUnit; Rust test	Critical
Integration Tests	DB interactions; rule persistence; VPN lifecycle; WFP rule apply	flutter integration_test	Critical
UI Tests	Navigation; dialogs; accessibility; state change rendering	flutter integration_test; Semantics API	High
Platform Tests	Android API 26–latest; Windows 10/11; battery; OEM devices	Manual + automated device farm	High
Performance Tests	Startup time; memory; CPU; DB query speed; 500+ rules	Flutter DevTools; systrace; WPA	High
Security Tests	Rule enforcement; import fuzzing; secure storage; permissions	Manual + SAST + network capture	Critical

Level	Scope	Tooling	Priority
Regression Tests	All previously fixed defects; critical user journeys	Full CI suite on every release	Critical

Table 30 — Testing Strategy by Level

5. Release Management



Figure 8 — Release Stage Pipeline

6. Risk Register

Risk	Likelihood	Impact	Mitigation
Android OEM VPN compatibility issues	High	High	Test on Samsung, Xiaomi, Huawei; document known issues; provide workarounds in-app
VpnService killed by Android battery optimisation	High	High	Use foreground service; request battery exemption; document user instruction
Windows WFP driver compatibility	Medium	High	Test on Win10 22H2 and Win11; sign all drivers; graceful degradation if WFP unavailable
Blocklist false positives breaking apps	Medium	High	User allow-list always overrides blocklist; easy domain reporting; conservative defaults
DNS provider outage causes connectivity loss	Medium	High	Auto-fallback to system DNS with user warning; offline cached blocklist still active
User confusion between firewall and VPN	High	Medium	Language guide; onboarding wizard; contextual help tooltips; user testing
Schema migration failure on app update	Low	High	Automated migration tests; backup before migration; documented recovery path
Export file tampering on re-import	Low	Medium	Checksum and schema validation; reject on any error; no partial imports permitted

Table 31 — Risk Register

7. Accessibility Standards

Standard	Requirement	Testing Method
Colour Contrast	Minimum 4.5:1 for normal text; 3:1 for large text	Automated contrast checker and manual review
Focus Indicators	Clearly visible focus ring on all interactive elements (Windows)	Manual keyboard navigation test
Content Descriptions	All icons and images have a non-empty meaningful content description	TalkBack / Narrator automated scan
Touch Targets	All interactive elements minimum 48×48dp (Android)	Automated size checker in CI pipeline
Text Scaling	Layout intact at 200% system font scale	Manual test at 200% on each platform
Reduced Motion	No auto-playing animations when system reduced-motion is enabled	Manual test with setting active
Error Identification	Errors describe what is wrong and what the user should do	Manual UX review of all error states
Status Announcements	Screen reader announces all protection status changes and profile switches	TalkBack / Narrator script test

Table 32 — Accessibility Standards

8. Post-V1 Future Roadmap

- Linux platform support using nftables / iptables integration
- macOS platform support using the Network Extension framework
- Browser companion extension for Firefox and Chromium
- Advanced conditional rule scripting with time-based and app-state conditions
- Community-maintained curated blocklist packs with one-click subscription
- Optional end-to-end encrypted profile sync between a user's own devices
- Optional enterprise policy import/export compatible with air-gapped environments
- Heuristic local network anomaly detection identifying unusual destination patterns
- Biometric app lock and encrypted local backup with user-controlled recovery
- Android home screen widget and Quick Settings tile for one-tap mode switching
- Full IPv6 support across all engines and rule types

9. Definition of Done

A feature is complete only when ALL of the following are verified:

- Functional implementation complete and code reviewed by a peer
- Unit tests written and passing
- Integration tests written and passing
- UI reviewed against the Beginner-First Language Guide
- Accessibility verified — WCAG 2.1 AA compliance confirmed
- Performance acceptable on reference hardware
- Documentation updated (API comments, SDES reference, developer notes)
- Security review completed for any new data access, permission, or network path
- Zero known critical or high-severity defects remain against this feature

10. Final Product Statement

Network Cloak is defined by a set of inviolable principles. Every future version, every new feature, and every architectural decision must continue to satisfy all of them. If a proposed change conflicts with any principle, the change must be redesigned — not the principle.

- Local-first — all data is processed on-device; cloud is never required
 - Privacy-first — no unnecessary data collection; no telemetry without explicit consent
 - Transparent — every decision is explainable in plain language to any user
 - User-controlled — the user makes all decisions; the application never acts autonomously
 - Cross-platform — consistent feature parity on Android and Windows
 - No mandatory infrastructure — no cloud account, subscription, or relay server required
 - No black-box behaviours — all rule evaluations are logged and auditable
 - Accessible — WCAG 2.1 AA compliance maintained across all platforms
 - Maintainable — clean architecture with enforced boundaries and comprehensive test coverage
-

End of Software Design & Engineering Specification — Volumes I–VII. This document defines the complete product vision, functional requirements, UI/UX specification, technical architecture, data architecture, security and privacy model, and development roadmap for Network Cloak version 1.0.

Recommended Companion Documents

To complete the implementation package, the following companion documents should be produced alongside this SDES:

1. Screen-by-screen wireframes and high-fidelity mockups for all platforms
2. Interactive UX flow specification covering all user journeys and edge cases
3. C4 Model architecture diagrams (Context, Container, Component, Code levels)
4. Sequence diagrams for all inter-module communications and platform channel calls
5. Full normalised database ER diagram with all foreign keys and constraints
6. State machine specifications for Firewall, Shield, Rule Engine, Temporary Rules, and Profiles
7. Flutter ↔ Native Platform Channel API contract specification
8. Android Engineering Specification (VpnService lifecycle and OEM compatibility matrix)
9. Windows Engineering Specification (WFP callout structure, service installation, and signing)
10. Design System Specification (tokens, component library, spacing, typography, and icons)
11. Comprehensive test plan and test cases (functional, integration, security, and performance)
12. STRIDE threat model diagram
13. Developer contribution guide (branching, review process, CI/CD, and versioning)
14. Release operations manual (signing, packaging, store submission, and rollback)
15. Privacy policy and security whitepaper aligned with the implemented application behaviour